

QUERYDIAGRAAMEVAL: Evaluating and Benchmarking LLM Diagram Generation under Noisy Conditions

Ted Zhang¹, Souvik Das¹, Kai Ni¹, Aditya Agrawal¹, Xin Guo¹, Denis Kochedykov¹

¹J.P.Morgan Chase & Co.
ce.zhang@jpmchase.com

Abstract

Chat assistants increasingly ask large language models to turn free-form user prose into structured, executable diagrams. These requests are noisy user-generated text: informal, hedged, jargon-mixed, incomplete, and often describing the target only indirectly—in workflow language rather than diagram vocabulary. Evaluation is hard because many distinct diagrams satisfy the same noisy request, making reference-based metrics too rigid where no ground truth exists, while reference-free LLM judges give weak, hard-to-reproduce diagnostics. We introduce QUERYDIAGRAAMEVAL (QDE), a reference-free protocol scoring diagram generation directly against raw, noisy queries via two levels: deterministic compilation gates (syntax, rendering, diagram-type correctness), then a semantic-fidelity layer where a multimodal auditor is grounded in type-specific structural facts parsed from the code. On 255 annotated predictions, QDE aligns strongly with holistic human ratings ($\rho = 0.715$), outperforming an ungrounded LLM judge. We also release MERMAIDBENCH, a source-grounded benchmark of 1,150 query–gold pairs over 11 diagram types, four query-difficulty tiers, and a multi-turn track, with the gold held fixed to isolate robustness to input-side noise. Furthermore, we conduct experiments on eight frontier-model and harnesses to show that our agentic framework reaches near-gold quality while failures concentrate in topology preservation, label grounding, and type-specific syntax.

1 Introduction

Large language models (LLMs) are increasingly asked to turn *user-generated natural-language requests* into structured, executable artifacts (Naveed et al., 2024). Diagram generation is a particularly demanding case: in chat assistants such as GitHub Copilot, Claude Code, and ChatGPT, users routinely ask models to turn free-form prose into flowcharts, sequence diagrams, ER diagrams, Gantt

charts, and other visual specifications (Zala et al., 2024). In production these requests are *noisy user text*: informal, hedged, jargon-mixed, incomplete, and often describing the target only *indirectly*—in workflow language rather than diagram vocabulary. The gap between what the user typed and what the diagram must depict is precisely the noisy-text challenge, but the downstream artifact is now a compilable typed visual program.

Evaluation methods inherited from code-generation benchmarks such as HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and SWE-bench (Jimenez et al., 2024) do not transfer: diagrams compile into visual artifacts rather than executable behaviour, and many structurally distinct diagrams may all be valid answers to a single noisy query, making exact-match and pixel-similarity metrics against a gold reference brittle (Liang and You, 2025). Deployment is also inherently *reference-free*—only the user’s noisy query is available at inference—so reference-based metrics are unusable for online evaluation, A/B testing, or guardrails on user traffic. Silent semantic failures (a rendered flowchart with a reversed arrow or a missing service) are far more costly than syntax errors, yet “does it render?” CI checks miss them and human review does not scale.

We study these challenges in Mermaid¹, a widely deployed text-to-diagram language whose typed structure (flowchart nodes/edges, sequence participants/messages, ER entities/relations, etc.) enables automatic checking while still requiring the evaluator to judge intent expressed in *unconstrained, potentially noisy* natural language.

We introduce a reference-free evaluation protocol, QUERYDIAGRAAMEVAL (QDE), and a Mermaid benchmark, MERMAIDBENCH, that jointly stress-test diagram generation *under noisy user text*. QDE first applies deterministic compilation

¹<https://mermaid.js.org/>

gates (syntax, render, type), then extracts type-specific structural facts from the code and passes them—together with the raw user query and rendered image—to a multimodal auditor that performs a fact-anchored requirement audit; $k=5$ self-consistent samples are averaged. MERMAID-BENCH contains 1,150 tasks derived from 230 expert-verified real-world diagrams. Each source is fanned out into four single-turn *query tiers*—clean explicit spec (T1), adversarial noise and hedging (T2), workflow phrasing with no diagram terms (T3), indirect scenario (T4)—plus a multi-turn refinement track, with the gold diagram held fixed across tiers so scores isolate robustness to *input-side noise* across all 11 stable Mermaid types.

Across eight frontier configurations, QDE correlates strongly with human holistic ratings ($\rho=0.715$). Query indirection is the dominant difficulty axis (-1.8 T2 $\rightarrow$ T3, -2.1 T3 $\rightarrow$ T4), confirming that current LLMs remain brittle when user text drifts from clean specs, and residual failures are semantic—missing/hallucinated topology, label mismatches, type-specific slips—rather than syntactic.

Contributions.

1. QUERYDIAGRAMEVAL, a reference-free two-level protocol for scoring LLM diagram generation against *raw user queries* ($\rho=0.715$ with human ratings).
2. MERMAIDBENCH, a source-grounded benchmark of 1,150 tasks over 11 Mermaid types, four *noise-controlled* query tiers, and a multi-turn refinement track.
3. MERMAIDAGENT, a ReAct-based tool-augmented reference architecture evaluated across the full noise spectrum.

2 Related Work

Diagram evaluation. Prior diagram-quality metrics fall into three families. *Pixel-level similarity*, used by AutomaTikZ (Belouadi et al., 2024) (CLIP-Score) and DiagrammerGPT (Zala et al., 2024) (raster outputs), measures visual closeness to a reference image, ignoring whether the underlying code is structurally correct or grammar-valid. *Reference-based graph metrics* address this by parsing diagrams into graphs: DiagramEval (Liang and You, 2025) computes node- and path-alignment F1 between predicted and gold graphs, while DiagramAgent (Wei et al., 2024) reports task-specific checks against gold templates. *LLM-*

as-judge approaches such as MT-Bench (Zheng et al., 2023) and the Mermaid-specific MermaidSeqBench (Shbita et al., 2025) score outputs holistically but inherit verbosity, position, and self-enhancement biases. All three families either require a gold reference, degenerate on list-oriented types (Gantt, timeline, journey, quadrant), or expose the final score to unconstrained judge bias. QUERYDIAGRAMEVAL (§3) closes this gap by anchoring a multimodal LLM judge with type-specific structural facts deterministically parsed from the candidate code, enforcing Level 1 compilation gates, and aggregating $k=5$ self-consistent samples—producing a *reference-free* score that correlates with human ratings at $\rho=0.715$.

Benchmarks for structured code generation.

Program-generation benchmarks such as HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), DS-1000 (Lai et al., 2023), and SWEbench (Jimenez et al., 2024) grade programs through execution, providing no signal for visual or structural artefacts. On the diagram side, ChartQA (Masry et al., 2022) and PlotBench (Zhao et al., 2025) target charts rather than typed diagrams; AutomaTikZ covers a single TikZ format with no difficulty control; MMCode (Li et al., 2024) targets general multimodal programming; DiagramAgent’s DiagramGenBench is synthetically constructed across eight categories without source-grounded golds; MermaidSeqBench covers only sequence diagrams (one of 11 Mermaid types) with 132 samples. Synthetic construction approaches like Self-Instruct (Wang et al., 2023) and Evol-Instruct (Xu et al., 2024) risk circular bias when benchmark artefacts are pure LLM self-play. MERMAIDBENCH (§4) fills this gap with 340 test tasks (1,150 total) spanning 11 Mermaid types, four query tiers, and a multi-turn track, with gold diagrams distilled from real-world images and human-audited. None of these prior protocols are directly deployable in the reference-free regime that production diagram assistants actually face; MERMAIDBENCH and QDE combine broad typed Mermaid coverage with source-grounded construction and reference-free scoring, making them suitable both as a research benchmark and as an in-production quality gate.

3 QueryDiagramEval

Diagram code admits multiple valid realizations for any given query, making reference-based

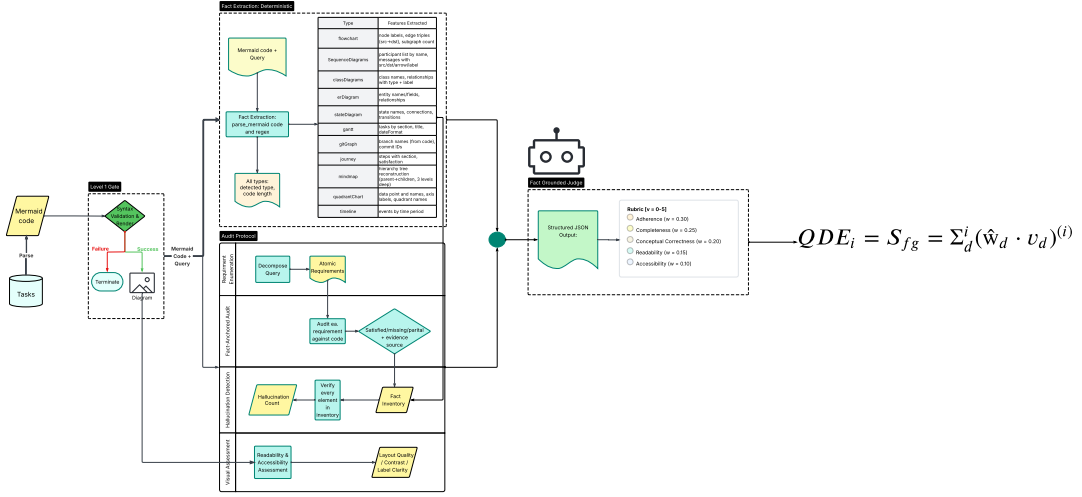


Figure 1: The two-level QUERYDIAGRAMEVAL (QDE) protocol. Level 1 runs a deterministic parser and renderer plus a type check whose outcomes act as hard gates. Level 2 extracts type-specific structural facts from the candidate code and provides them, together with the query and rendered image, to a multimodal auditor that produces a fact-grounded rubric score. QDE is reference-free: only the query and predicted code are required.

exact-match metrics unreliable. Relying solely on an LLM-as-judge, in contrast, conflates semantic assessment with structural and compilation correctness, sacrifices reproducibility, and inherits known judge biases (verbosity preference, self-enhancement; Zheng et al., 2023). We therefore propose QUERYDIAGRAMEVAL (QDE), a *reference-free, two-level* protocol (Figure 1) that combines **compilation verification** (§3.1)—deterministic checks for syntax validity, render success, and type correctness, enforced as hard gates—with **fact-grounded semantic assessment** (S_{fg} , §3.2)—a multimodal LLM auditor anchored by deterministic structural facts extracted from the candidate code, aggregated by $k=5$ self-consistent samples.

Given a user query q and predicted Mermaid code $\hat{y}$, the QDE score is

$$QDE = S_{fg} \quad \text{subject to Level 1 hard gates.} \quad (1)$$

This design ensures that (i) structurally broken code cannot achieve high scores regardless of auditor output; (ii) the auditor operates on verifiable structural evidence rather than surface impressions; and (iii) Level 1 metrics (render rate, type accuracy) are fully deterministic and independently reportable.

3.1 Level 1: Compilation Verification

The mermaid code block is extracted from the response and parsed by a type-aware AST parser supporting all 11 stable Mermaid types. The parser

identifies the declared type and extracts typed structural elements (nodes, edges, subgraphs, participants, tasks, commits, entities, etc.). Valid code is rendered to SVG and PNG.

Three binary signals—*parse pass rate*, *render success rate*, and *type accuracy*—are recorded as independent metrics and enforced as hard gates. If any of the binary signals fails the candidate mermaid code is rejected therefore preventing QDE from generating a unreliable score, especially if the mermaid code contains syntax errors that make rendering impossible.

3.2 Level 2: Fact-Grounded Semantic Assessment

Type-specific fact extraction. Pure LLM-as-judge evaluation operates on surface impressions of the code and image, without verified structural information; the auditor may hallucinate elements, miscount entities, or overlook structural deficiencies. QDE grounds the auditor with deterministic structural facts extracted from the candidate code. A type-aware parser produces a fact summary tailored to the diagram type: flowchart nodes/edges/subgraphs, sequenceDiagram participants and messages with labels, classDiagram classes and typed relationships, erDiagram entities and relationships, stateDiagram states/transitions, gantt tasks/sections, gitGraph branches/commits, journey steps with satisfaction scores, mindmap hierarchy trees, quadrantChart data points and axes,

and timeline events (Appendix B, Table 9). These facts are deterministic and verifiable—they represent ground truth about what the code actually contains.

Fact-anchored requirement audit. A multi-modal auditor receives four inputs: the user query q , the Mermaid source $\hat{y}$, the rendered PNG, and the structural fact summary. It performs a five-step chain-of-thought assessment: (i) decomposes q into atomic requirements; (ii) classifies each as satisfied, missing, or partial by cross-referencing the structural facts and its own semantic analysis, tagging each decision with its evidence source (structural_fact, semantic, or image); (iii) flags hallucinated elements; (iv) scores readability and accessibility from the rendered image; and (v) returns a JSON object with per-requirement statuses, error counts, and five 0–5 dimension scores. The complete system prompt, structural-fact template, and per-task user message appear in Appendix D.

Rubric aggregation and self-consistency. The five dimension scores $v_d \in \{0, \dots, 5\}$ are aggregated with fixed weights $\hat{w} = (0.30, 0.25, 0.20, 0.15, 0.10)$ over (adherence, completeness, conceptual correctness, readability, accessibility) and scaled to $[0, 100]$ as shown in equation Equation 2. Adherence and completeness dominate because diagram-generation failures are usually semantic. To reduce stochastic judge variance we sample the audit $k=5$ times and take the mean:

$$S_{\text{fg}}^{(i)} = 20 \sum_d \hat{w}_d v_d^{(i)}. \quad (2)$$

On the validation pool ($N=255$), the mean inter-sample standard deviation is 2.4 points on 0–100 (median spread 4.0); only 14% of items exceed $\sigma=5$. The per-task $\bar{\sigma}_k$ is reported alongside model scores (Table 2) as a judge-stability diagnostic.

Why reference-free? Reference-based structural metrics (e.g., graph F1 against gold code) penalise valid alternative realizations—paraphrased labels, reordered nodes, equivalent subgraph decompositions. On our validation pool, a reference-based deterministic audit achieves only $\rho=0.462$ with human ratings, versus $\rho=0.715$ for the fact-grounded reference-free auditor. More practically, reference-free evaluation is a strict prerequisite for online deployment (CI/CD, reward models, quality gates), where gold references do not exist.

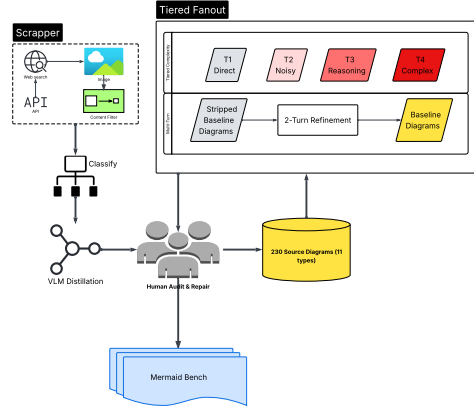


Figure 2: MERMAIDBENCH construction pipeline. Real-world diagrams are collected, distilled to Mermaid via a VLM using a describe-then-extract protocol, expert-audited and repaired, then fanned out into four query tiers plus a multi-turn track, quality-gated, and split into dev/test sets.

4 MermaidBench

MERMAIDBENCH is designed to be *source-grounded* (anchored to real-world diagrams, not LLM self-play), *type-diverse* (11 Mermaid types), *difficulty-controlled* (query-side only), and *contamination-resistant* (canary identifiers).

Source collection and VLM-assisted authoring.

We source 323 candidate technical diagrams from academic papers, technical documentation, and image search. An LLM-first classifier retains true technical diagrams and predicts a canonical Mermaid type; cheap structural gates (aspect ratio, minimum resolution, text-region count) reject obvious non-diagrams. After quality filtering and type balancing, 230 sources enter the authoring pipeline. For each source, a VLM produces baseline Mermaid code via a *describe-then-extract* protocol: it first produces a detailed textual description of the diagram’s components, relationships, and layout, then generates Mermaid targeting the predicted type conditioned on both image and description. Distillation runs in a 3-retry loop with rendering-based validation and *fidelity-guided* feedback: type-specific structural checks (e.g., node/edge counts for flowcharts, participant/message counts for sequence diagrams) supply targeted feedback on failures.

Expert audit and repair. All 230 baselines undergo expert audit for type fidelity, element coverage, structural accuracy, hallucinations, and label faithfulness; annotators repair every identified error. The VLM draft functions as a productivity accelerator rather than the ground truth: 100% of released baselines are human-verified.

Tier fan-out and multi-turn track. Every verified baseline is converted into four single-turn query tiers plus one multi-turn task. **T1 (Direct)** names diagram elements explicitly using Mermaid terminology; **T2 (Adversarial)** introduces noise, distractors, and hedging around the same requirements; **T3 (Reasoning)** uses workflow or business phrasing with no Mermaid jargon; **T4 (Complex)** describes an indirect scenario requiring inference of diagram structure. The gold code is held fixed across tiers, isolating query-comprehension difficulty from gold-side variation. Multi-turn tasks start from a stripped-down baseline (edges and styling removed); the user acknowledges the simplified diagram and then requests specific improvements toward the full baseline, isolating iterative-refinement capability. A representative tier progression for a single source diagram appears in Appendix A, Figure 4.

Contamination resistance. We inject unique canary identifiers into gold node IDs during baseline construction: each baseline receives three mutations following `XQV{NNNN}_{shortid}` (globally unique per run), and a quality gate verifies that the mutated code still renders. Because the tier queries are authored from the canary-mutated gold, a canary token can legitimately surface in the query itself (and, for multi-turn tasks, in the simplified diagram shown to the model), so a model can reproduce a canary simply by *copying* it from its own input. We therefore do *not* treat mere canary appearance as memorization. Contamination is flagged only by a *hidden-canary* match—a canary that appears in a generation but in *none* of the model-visible inputs (query plus any conversation context). The public dev split has canaries reverted to their original identifiers before release, while the test split retains them for this probe. Across all evaluated systems we observe *zero* hidden-canary matches, i.e., no detectable memorization of test golds; full procedure and counts are in Appendix A.1.

Split	Base	Single-turn	Multi-turn	Total
Dev	162	648	162	810
Test	68	272	68	340
All	230	920	230	1,150

Table 1: Dataset scale. Full per-type composition appears in Appendix A, Table 7.

Quality gates and splitting. Baselines must pass three automated gates before release: (i) parses and renders via the pinned Mermaid.js runtime; (ii) canary survival; and (iii) readability thresholds—label overlap < 0.08 , whitespace ratio $\in [0.30, 0.85]$, aspect ratio $\in [0.3, 3.0]$, plus type-specific extractability. The fan-out produces $230 \times 4 = 920$ single-turn tasks and 230 multi-turn tasks (1,150 total). A stratified 70/30 dev/test split (810/340) is computed with a fixed seed and an anti-leakage constraint that prevents variants of the same source from crossing splits.

5 MermaidAgent

MERMAIDAGENT is a tool-augmented language agent that generates Mermaid diagrams from natural-language queries under the Reasoning-and-Acting paradigm (Yao et al., 2023). It serves both as a competitive system on MERMAIDBENCH (§6) and as a reference architecture for grounding LLM generation in deterministic Mermaid tooling. The agent separates *planning* (what to draw), *generation* (how to draw it), and *evaluation* (whether the draft is correct). Diagram generation from natural language poses three compounding challenges: (i) Mermaid’s strict, type-specific syntax means a single malformed token blocks rendering; (ii) semantic correctness cannot be verified by a rule-based check; and (iii) different diagram varieties impose fundamentally different structural constraints. Each LLM-driven stage is therefore grounded by deterministic tools that supply authoritative syntax and validate output.

Planner. Two sequential LLM calls, each constrained to a structured output schema, produce first an *intent analysis* (closed-vocabulary diagram type, estimated complexity, subject domain, ranked key requirements, detected ambiguities, style constraints) and then a *diagram plan* (structural outline, key nodes/entities, edges/relationships, styling approach, and a justification of complexity). The closed type vocabulary bounds the search space; the plan is injected into the generation prompt as a soft constraint, preserving generative flexibility

while maintaining intent alignment.

Tool registry. Four tools support the agent. Two are knowledge tools consulted once per request: $\mathcal{T}_1$ (documentation) returns a 700–1,000-character authoritative syntax summary for the target type; $\mathcal{T}_2$ (design principles) injects expert guidelines along four axes—general clarity/consistency, type-specific rules, styling, and complexity-level element-count targets. Two are action tools invoked inside the ReAct loop: $\mathcal{T}_3$ (repair) provides two-stage syntax validation, structure analysis, and rule-based fix suggestions; $\mathcal{T}_4$ (critic) performs deterministic multi-dimensional evaluation and returns syntax validity, type correctness, a requirement-coverage score s_{cov} , structural-completeness flags, and a syntax-weighted alignment score $s_{\text{align}} \in [0, 1]$. Full tool schemas, prompts, the coverage/alignment formulas, and the approval predicate ($\text{syntax_valid} \wedge \text{type_match} \wedge |\text{design_issues}| \leq 1 \wedge s_{\text{cov}} \geq 0.7$) are reproduced in Appendix E (Figure 5).

ReAct loop. The core loop runs up to $N_{\text{max}} = 5$ Thought–Act–Observe iterations. Iteration 1 composes a generation prompt from the query, intent, plan, syntax reference, and design principles, and applies structural validation immediately; on failure a focused syntax-only sub-routine constrained to minimal edits fires before critique, preventing a trivially invalid draft from consuming a full iteration. Iterations 2– N_{max} consume critic feedback, emit a structured repair plan, and apply a full repair whose prompt includes the plan and the first 1,000 characters of the syntax reference as a grounding guard. The loop exits early when the critic returns approved = true with $s_{\text{align}} \geq 0.5$; otherwise the last candidate is returned with a warning. Every step is trace-logged; a 2–3-sentence caption is produced at termination.

6 Experiments

6.1 Setup

We report two studies over disjoint data partitions. A *metric-validation study* (§6.2) tests QDE–human alignment on a quality-stratified pool of 255 predictions with three human ratings each (inter-annotator Krippendorff’s $\alpha=0.797$ on holistic ratings; Krippendorff, 2011; full annotator, training, blinding, rubric, adjudication, and per-dimension agreement details in Appendix C). A *model-comparison study* (§6.3) evaluates eight configurations on the

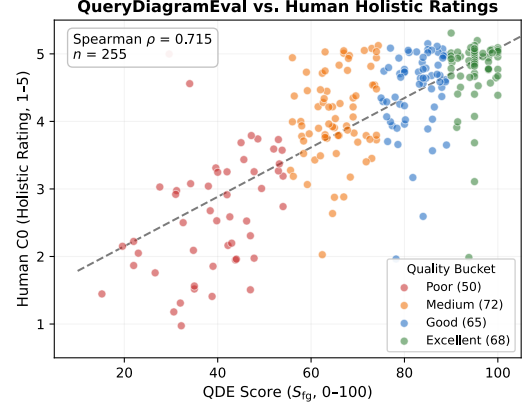


Figure 3: QDE vs. human holistic ratings (C0) on the validation pool ($n=255$). Points are coloured by quality-stratification bucket. Spearman $\rho = 0.715$.

340-task MERMAIDBENCH test set (272 single-turn + 68 multi-turn), scoring every model output against its authored query. Direct configurations use standard prompting; reasoning configurations enable extended thinking; agentic configurations use tool-augmented iterative refinement—the Mermaid-specific MERMAIDAGENT (§5) and a Mermaid port of DiagramAgent (Wei et al., 2024) sharing the same pinned Mermaid backend.

6.2 Metric Validation

The validation pool combines real model outputs ($\sim 60\%$) with controlled gold-code degradations ($\sim 40\%$). We degraded the gold code with label scrambling, partial omits, edge deletion and node hallucination. The original benchmark query is always preserved. Predictions are then bucketed by automatic QDE score into four quality bands before annotation so that no single quality band dominates the correlation estimate. Because this stratification deliberately spans the metric’s range, we also report correlation on the naturally-distributed real-output subset alone ($\rho=0.444$, $n=134$; attenuated by ceiling concentration), which remains positive and significant in Section 7 the Discussion.

QDE achieves the strongest alignment with human ratings: the fact-grounded S_{fg} reaches $\rho=0.715$ (Table 3), compared with $\rho=0.668$ for an ungrounded chain-of-thought judge (+0.047; improvements on 9 of 11 types) and $\rho=0.462$ for a reference-based structural score (GRAPHF1, a deterministic node/edge-set F1 against the gold code adapted from the graph-based diagram metric of Liang and You, 2025; full specification, including label normalization, node alignment, edge-direction handling, and per-type behavior on non-

Model	QDE by Tier				QDE Overall	$\bar{\sigma}_k$	MT QDE	Render %
	T1	T2	T3	T4				
GPT-5.4	68.9	76.8	74.6	70.8	75.1	1.3	84.2	87.1
GPT-5.4 (reasoning)	81.2	75.0	76.8	65.6	76.7	1.1	84.8	85.9
Claude Opus 4.6	82.6	83.7	87.0	83.0	85.4	1.3	90.9	97.1
Claude Opus 4.6 (reasoning)	85.9	85.7	85.9	84.2	85.4	1.6	79.7	98.0
DiagramAgent w/ GPT-5.4	79.6	81.8	74.7	73.8	77.5	1.6	77.7	94.4
DiagramAgent w/ Claude Opus 4.6	80.1	83.2	77.7	76.5	81.3	1.3	87.7	95.2
MermaidAgent w/ GPT-5.4	77.0	70.8	69.0	77.6	73.6	1.6	86.7	88.7
MermaidAgent w/ Claude Opus 4.6	88.4	87.9	85.0	82.8	86.7	1.3	89.4	96.5
<i>Gold baseline</i>	88.9	90.1	87.9	87.8	89.5	1.5	95.6	100.0

Table 2: Main results on the MERMAIDBENCH test set ($n=340$). QDE is on 0–100. MT = multi-turn. $\bar{\sigma}_k$ = mean per-task standard deviation across $k=5$ judge samples. Type accuracy (not shown): $\geq 86.8\%$ for all models. Per-type breakdowns and tier deltas appear in Appendix F.

Metric variant	Level	Spearman ρ
Graph F1	Ref.	0.462
S_{cot} (vanilla judge, no facts)	Ablation	0.668
S_{fg} (fact-grounded, $k=5$)	L2 (Semantic)	0.715

Table 3: Metric correlation with human holistic ratings (C0) on the validation pool ($N=255$). Level 1 binary metrics are hard gates rather than continuous predictors. 95% CI for S_{fg} : [0.640, 0.778].

graph diagrams, in Appendix F.3). The largest gains occur on structurally rich types such as class diagrams (+0.160), ER diagrams (+0.140), and journey diagrams (+0.093), where deterministic facts help the auditor count entities and relations reliably. Decomposing by rating dimension, semantic dimensions align most strongly with humans—adherence ($\rho=0.699$), conceptual correctness (0.654), completeness (0.582)—while visual dimensions are weaker but still positive (readability 0.316, accessibility 0.256). Per-bucket and dimension-wise tables appear in Appendix F.2.

6.3 Main Results

Table 2 reports the primary comparison. All 340 gold codes pass Level 1 gates and receive mean QDE = 89.5 (median 92.0; per-type range 79.4–93.8), confirming that the metric assigns high scores to known-correct code while retaining sensitivity. Because QDE is reference-free, a model can occasionally exceed the gold baseline on individual slices if it produces cleaner, more judge-amenable syntax than the human author; the gold row should therefore be read as a strong human reference point rather than a hard upper bound.

Three patterns stand out. **(1) Agentic refinement helps when paired with the strongest base model.** MERMAIDAGENT with Claude Opus 4.6 achieves the highest overall QDE (86.7) while

maintaining 96.5% render success, whereas the same scaffold on GPT-5.4 underperforms direct prompting (73.6 vs. 75.1)—suggesting that iterative repair loops magnify base-model quality rather than substituting for it. All three Claude configurations achieve $\geq 96\%$ render success. **(2) Reasoning has mixed effects.** GPT-5.4 reasoning slightly exceeds direct (76.7 vs. 75.1), primarily from a T1 boost (+12.3), but degrades on T4 (−5.2). For Claude Opus 4.6, reasoning matches direct on aggregate (85.4 vs. 85.4) with 98.0% render success, indicating that extended thinking does not degrade Mermaid generation but provides little marginal benefit at the frontier. **(3) Multi-turn dialogue often helps direct models.** GPT-5.4 direct improves from 72.8 single-turn to 84.2 multi-turn (+11.4), and MERMAIDAGENT/GPT-5.4 gains +13.1, consistent with multi-turn decomposing complex requirements into smaller edits. The DiagramAgent scaffold and Claude Opus 4.6 reasoning are exceptions (+0.2 and −5.7 respectively), highlighting that the multi-turn advantage is configuration-dependent.

Query indirection remains the primary difficulty axis. On average across models, QDE drops −1.8 from T2 to T3 and −2.1 from T3 to T4 as queries shift from explicit to indirect phrasing, while T1→T2 is essentially flat (+0.2)—adversarial noise alone is comparatively benign. Full tier deltas and per-type scores are in Appendix F.4 and F.1.

7 Discussion

Error taxonomy. Human error tags from repairing gold diagrams and rating model outputs (Appendix F.6) show that dominant failure modes are structural rather than syntactic: missing edges (22.6%), missing nodes (19.0%), layout/readability

(15.1%), label mismatches (14.8%), hallucinated nodes (14.0%), over-specification (13.7%), and factual errors (12.0%). Syntax and type errors are comparatively rare among frontier models but remain important because they render outputs unusable. This supports QDE’s design: deterministic Level 1 gates catch hard compilation failures, while fact-grounded semantic auditing exposes topology and label errors that “render success” alone cannot see.

Input-side verbosity leniency. Because every tier of a given baseline shares the same gold code, gold-diagram QDE scores across tiers isolate a pure evaluator effect. We observe an *input-side* verbosity leniency: verbose T2/T4 prompts sometimes receive higher S_{fg} than concise T3 paraphrases for the same gold, apparently because the auditor resolves ambiguity in favour of the candidate when requirements are diluted with distractors. In a matched comparison across 68 baselines, T4 receives higher QDE than T2 in 60% of cases ($\Delta QDE = +8.9$), concentrated in completeness, readability, and accessibility dimensions. This complements known output-side verbosity bias in LLM judges (Zheng et al., 2023) and motivates future query-length normalization; absolute QDE scores should be compared within-tier.

Self-enhancement bias. Because Claude Opus 4.6 is both the default judge and a candidate generator, self-preference is a natural concern. Indirect evidence argues against a severe effect: (i) the human-authored gold baseline outscores every LLM-generated system (89.5); (ii) Claude direct and Claude reasoning receive identical aggregate QDE (85.4 vs. 85.4); (iii) on the shared DIAGRAMAGENT scaffold the Claude variant exceeds the GPT-5.4 variant by only 3.8 points, consistent with their respective render-success rates; and (iv) among three fact-grounded judges evaluated against human holistic ratings, Claude attains the highest correlation ($\rho=0.715$ vs. GPT-5.2 $\rho=0.595$ and GPT-4.1 $\rho=0.568$). Direct evidence: re-scoring a stratified random sample of 175 (model, task) pairs with GPT-4.1 as an alternative fact-grounded judge yields model-level Kendall $\tau=0.905$ (Spearman $\rho=0.964$) with our main results; the top four systems retain their exact ranking (Appendix F.5). GPT-4.1 is systematically more lenient by 6–11 points, but the relative ordering that drives our conclusions is judge-invariant.

Implications. Because QDE is reference-free at inference time, it is directly deployable as (a) an offline benchmark metric, (b) an online CI/CD quality gate for LLM diagram assistants, and (c) a reward signal for fine-tuning diagram agents on natural queries where no gold exists. The Level 1 gates provide interpretable diagnostics—parse, render, and type accuracy—that decouple structural quality from semantic quality when tracking model progress.

Evaluation Datasets and Validation Pool The main results and the metric-validation results use different partitions and should not be conflated (Table 4). The validation pool tests QDE–human alignment across a broad quality range, whereas the held-out test set is used to compare model configurations on natural benchmark tasks.

	Validation pool	Test set
Purpose	Metric calibration	Model comparison
Section	\$6.2	\$6.3
Size (N)	255	340
Split	dev (annotation)	test
Prediction source	Real + degraded gold	Model outputs only
Query–gold pairing	Original (preserved)	Original (preserved)
Human ratings	✓ (3/task)	—

Table 4: The two evaluation datasets. They are disjoint and serve complementary roles.

Validation-pool composition. Table 5 lists the composition. Real predictions provide medium-to-high-quality outputs; controlled degradations create known failure modes. Perturbations are applied only to the candidate diagram code—the original query is preserved so that the pool tests whether QDE penalises diagrams that fail to satisfy a valid request without introducing query–gold mismatch as a confound.

	Source	Approx. share
Real predictions	GPT-4.1 (T1–T4 + MT)	~60%
Degraded (gold-side)	label_scramble	~19%
	node_hallucination	~10%
	edge_deletion	~9%
	partial_omit	~6%
	syntax_break	~3%

Table 5: Validation-pool composition. All items pair the original benchmark query with either a real model output or a perturbed gold diagram.

Stratified vs. naturally-distributed sampling.

Because the pool is quality-stratified—and the synthetic degradations are constructed to span QDE’s range—correlation on the full pool partly reflects the design’s deliberate coverage of the metric’s scale rather than an unbiased draw of naturally occurring outputs. To separate these effects, Table 6 recomputes $\rho(\text{C0}, \text{QDE})$ on three subsets: the full stratified pool, the *naturally-distributed* real model outputs only (no synthetic perturbation, not selected to span the metric), and the degraded-gold items only. As expected, the degraded subset carries most of the between-band signal ($\rho=0.839$) since perturbations create wide, easily separable quality gaps, while the natural real-output subset is lower ($\rho=0.444$). Crucially, the natural subset is strongly ceiling-concentrated (mean C0 = 4.46 vs. 4.08 pool-wide), so its reduced ρ is largely range restriction rather than metric failure—the same ceiling-compression effect documented in the appendix Table 17—and QDE still tracks human judgment positively and significantly on naturally occurring outputs (95% CI excludes zero). We report both numbers rather than the stratified figure alone. Two caveats remain: the natural subset covers only GPT-4.1 outputs (human ratings were not collected for the other Table 2 systems), and its ceiling concentration limits the achievable within-sample ρ ; a larger natural-distribution annotation spanning all evaluated systems is left to future work.

Subset	n	mean C0	$\rho(\text{C0}, \text{QDE})$
Full stratified pool	255	4.08	0.715
Real outputs only (natural)	134	4.46	0.444
Degraded gold only	121	3.66	0.839

Table 6: Correlation under stratified vs. naturally-distributed sampling. The natural (real-output) subset is not selected to span the metric and is ceiling-concentrated, which restricts its within-sample ρ ; the correlation nonetheless remains positive and significant (95% CI [0.296, 0.570]).

8 Conclusion

We presented QUERYDIAGRAEVAL, a reference-free two-level protocol for diagram-code evaluation that combines deterministic compilation gates with fact-grounded multimodal semantic assessment, and MERMAIDBENCH, a 1,150-task source-grounded benchmark spanning 11 diagram types, four query tiers, and a multi-turn track. Human validation shows strong QDE-to-human alignment

($\rho=0.715$), improving over ungrounded LLM-as-judge ($\rho=0.668$) and reference-based structural scoring ($\rho=0.462$). Experiments on eight frontier configurations show that agentic refinement with Claude Opus 4.6 approaches the human-authored gold baseline, while residual failures concentrate in topology preservation, label grounding, and type-specific syntax. Because QDE requires no reference at inference time, it is immediately deployable as an evaluator for diagram assistants and as a reward signal for fine-tuning diagram agents. We hope these resources support progress on reliable diagram generation and, more broadly, on evaluation methods for structured visual code in production LLM systems.

Limitations

MERMAIDBENCH covers Mermaid only; results may not transfer directly to PlantUML, DOT, or TikZ, though Mermaid is the most widely adopted LLM-native diagram language and its typed grammars produce structured, parseable output particularly suited to fine-grained automated evaluation. We cover Mermaid’s 11 stable core types and exclude the trivial pie type, five beta-tagged types with unstable syntax (sankey-beta, xychart-beta, block-beta, packet-beta, architecture-beta), plugin-based extensions (C4Context, zenuml), kanban (added post-corpus collection), and requirementDiagram (extremely niche). The source corpus is English-heavy and web-scraped, so domain skew is possible; we release only distilled Mermaid golds and metadata where licensing prohibits redistribution of raw images. QDE Level 1 is fully deterministic, but Level 2 still uses a multimodal LLM at inference; grounding and $k=5$ self-consistency reduce variance and judge bias but do not eliminate them, and the input-side verbosity leniency described in §7 means absolute QDE scores are not perfectly comparable across tiers.

Disclaimer

This paper was prepared for informational purposes in part by the Machine Learning Center of Excellence group of JPMorgan Chase & Co. and its affiliates (“JP Morgan”) and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the information contained

herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. [Program synthesis with large language models](#). *Preprint*, arXiv:2108.07732.
- Jonas Belouadi, Anne Lauscher, and Steffen Eger. 2024. [AutomaTikZ: Text-guided synthesis of scientific vector graphics with TikZ](#). In *Proceedings of the Twelfth International Conference on Learning Representations*.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [SWE-bench: Can language models resolve real-world GitHub issues?](#) In *Proceedings of the Twelfth International Conference on Learning Representations*.
- Klaus Krippendorff. 2011. [Computing Krippendorff’s alpha-reliability](#). *Departmental Papers (ASC)*.
- Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-Tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. [DS-1000: A natural and reliable benchmark for data science code generation](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18319–18345. PMLR.
- Kaixin Li, Yuchen Tian, Qisheng Hu, Ziyang Luo, Zhiyong Huang, and Jing Ma. 2024. [MMCode: Benchmarking multimodal large language models for code generation with visually rich programming problems](#). *Preprint*, arXiv:2404.09486.
- Chumeng Liang and Jiaxuan You. 2025. [Evaluating LLM-generated diagrams as graphs](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 12678–12690, Suzhou, China. Association for Computational Linguistics.
- Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. [ChartQA: A benchmark for question answering about charts with visual and logical reasoning](#). In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279. Association for Computational Linguistics.
- Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2024. [A comprehensive overview of large language models](#). *Preprint*, arXiv:2307.06435.
- Basel Shbita, Farhan Ahmed, and Chad DeLuca. 2025. [MermaidSeqBench: An evaluation benchmark for NL-to-Mermaid sequence diagram generation](#). *arXiv preprint arXiv:2511.14967*.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. [Self-instruct: Aligning language models with self-generated instructions](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508. Association for Computational Linguistics.
- Jingxuan Wei, Cheng Tan, Qi Chen, Gaowei Wu, Siyuan Li, Zhangyang Gao, Linzhuang Sun, Bihui Yu, and Ruifeng Guo. 2024. [From words to structured visuals: A benchmark and framework for text-to-diagram generation and editing](#). *arXiv preprint arXiv:2411.11916*.
- Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. 2024. [WizardLM: Empowering large pre-trained language models to follow complex instructions](#). In *Proceedings of the Twelfth International Conference on Learning Representations*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *International Conference on Learning Representations (ICLR)*.
- Abhay Zala, Han Lin, Jaemin Cho, and Mohit Bansal. 2024. [DiagrammerGPT: Generating open-domain, open-platform diagrams via LLM planning](#). In *Proceedings of the Conference on Language Modeling (COLM)*.
- Yi Zhao, Zhen Yang, Shuaiqi Duan, Wenmeng Yu, Zhe Su, Jibing Gong, and Jie Tang. 2025. [PlotGen-Bench: Evaluating VLMs on generating visualization code from diverse plots across multiple libraries](#). *Preprint*, arXiv:2601.11525.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin,

Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph Gonzalez, and Ion Stoica. 2023. [Judging llm-as-a-judge with mt-bench and chatbot arena](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

A Additional MermaidBench Details

The main paper summarises MERMAIDBENCH at a high level. This appendix gives the supporting construction detail needed to reproduce the benchmark and interpret the aggregate statistics. Figure 4 illustrates the design principle that difficulty is introduced through the user query while the gold Mermaid program remains fixed for a source diagram, isolating query-understanding difficulty from variation in target-code complexity.

Tier generation. Every verified baseline participates in every tier. T1 directly names diagram elements; T2 introduces adversarial noise, hedging, and distractors around the same requirements; T3 uses workflow/business phrasing without Mermaid terminology; T4 describes an indirect scenario requiring structural inference. Multi-turn tasks begin from a stripped-down baseline (edges and styling removed) and ask for specific edits toward the full gold. The gold Mermaid code is held fixed across single-turn tiers so that scoring differences reflect query-comprehension difficulty rather than gold-side variation.

Quality gates. Baselines pass three gates before release: (i) syntax/render validation using a pinned Mermaid.js runtime; (ii) canary-token preservation (three $XQV\{NNNN\}_{\{shortid\}}$ mutations per baseline, verified after mutation); and (iii) readability thresholds—label overlap < 0.08 , whitespace ratio $\in [0.30, 0.85]$, aspect ratio $\in [0.3, 3.0]$ —plus type-specific extractability. Baselines failing any gate are repaired or excluded.

Source collection and VLM-assisted authoring.

Sources were collected from academic papers, technical documentation, and image search, then filtered by an LLM-first classifier and simple structural gates. From 323 verified source images, 230 quality-filtered, type-balanced sources entered distillation. For each source image, the authoring pipeline first asks a vision model (Claude Opus, selected after calibration against GPT-4o/GPT-5.2) to describe the components, relationships, and layout, then asks it to generate Mermaid conditioned on both the image and the description. A

rendering-based retry loop supplies fidelity-guided feedback when type-specific structural checks fail (e.g. node/edge count mismatches for flowcharts, participant/message mismatches for sequence diagrams).

Human audit and splitting. The VLM draft is not treated as ground truth. All 230 baselines undergo expert audit for type fidelity, element coverage, structural accuracy, hallucinations, and label faithfulness; identified errors are repaired before release. The 70/30 dev/test split is stratified by type and tier and includes an anti-leakage constraint that prevents variants of the same source from crossing splits.

Type	Base	T1–T4	MT	Dev	Test
flowchart	24	96	24	85	35
sequenceDiagram	28	112	28	100	40
classDiagram	24	96	24	85	35
stateDiagram	18	72	18	65	25
erDiagram	19	76	19	65	30
gantt	24	96	24	85	35
gitGraph	27	108	27	95	40
journey	9	36	9	30	15
mindmap	20	80	20	70	30
quadrantChart	15	60	15	55	20
timeline	22	88	22	75	35
Total	230	920	230	810	340

Table 7: Full dataset composition. MT = multi-turn tasks. Dev/test split is stratified by tier and type with source anti-leakage.

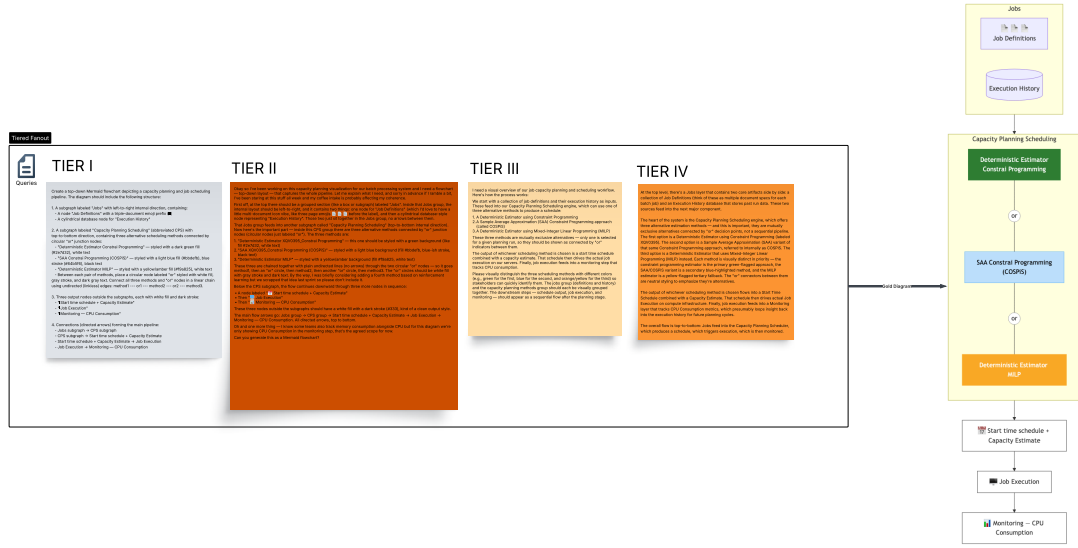


Figure 4: Example tier progression for a single source diagram. All four query tiers share the same gold Mermaid code; difficulty is controlled entirely on the query side. T1 names elements explicitly; T4 introduces noise and indirection.

A.1 Canary Contamination Audit

Canary identifiers guard against a model having memorized a test gold. Because a naive “canary appeared in the output” test is confounded by input copying, we specify the mechanism and detection precisely and report an audit over every evaluated system.

What is mutated, and where canaries can appear. Canaries are injected into *gold code node IDs*: during construction we rename three recurring identifiers per baseline to globally unique tokens $XQV\{NNNN\}_{\{shortid\}}$ (e.g. $User \rightarrow XQV0323_User$), rewriting every occurrence in the gold. The tier queries, however, are authored *from* the mutated gold, so canary strings routinely leak into the query text: in the 340-task test split, 183/340 (54%) golds carry at least one canary and 137/340 (40%) queries expose a canary token (full $XQV0323_User$ or bare $XQV0323$ prefix); restricted to canary-bearing golds, 63% expose the canary in the query. For multi-turn tasks, the simplified starting diagram supplied to the model likewise contains canary IDs. Consequently canaries are *not* hidden from the model, and their appearance in a generation is expected copying, not evidence of memorization.

Which canaries are hidden from the model. The only inputs guaranteed canary-free are the

public *dev* tasks, from which canaries are reverted to their original identifiers before release; the *test* split retains canaries so that a memorized test gold would surface a token the model was never shown. Thus a canary functions as a contamination probe *only* on the subset of test tasks where it does not already appear in the query or conversation context.

Detection procedure. For each generation we extract exact matches of the token pattern $XQV\langle 4 \text{ digits} \rangle_{\langle id \rangle}$ and, for every match, check whether the same canary prefix appears in any model-visible input for that task (the query text plus, for multi-turn tasks, the full conversation history). A match whose prefix is present in an input is a *copyable* emission (echoing a provided ID) and is discarded; a match whose prefix appears in *none* of the visible inputs is a *hidden-canary* match and is the only signal we count as possible contamination.

Audit result. We ran this test over all nine evaluated configurations on the test split (Table 8). Copyable emissions are common—up to 135 tasks per model—and concentrate on tasks whose query or multi-turn context already contains the canary; the two newest Claude configurations rewrite labels and emit no canaries at all. Critically, **no configuration produced a single hidden-canary match**: every emitted canary was traceable to a model-visible input. Under this criterion we detect no memorization of test golds. We note two limitations:

the probe covers only the three mutated IDs per gold (not all tokens), and it detects verbatim reproduction rather than paraphrased recall; a stronger guarantee would additionally strip the residual bare-prefix canary leakage from test queries, which we leave to a future benchmark revision.

System	Tasks w/ can.	Copyable tok.	Hidden tok.
Claude Opus 4.6	129	330	0
Claude Opus 4.6 (reasoning)	130	333	0
Claude Opus 4.7	0	0	0
Claude Opus 4.7 (reasoning)	0	0	0
GPT-5.4 (no reasoning)	135	339	0
GPT-5.4 (reasoning)	120	307	0
MERMAIDAGENT/GPT-5.4	60	156	0
DIAGRAMAGENT/GPT-5.4	130	321	0
DIAGRAMAGENT/Opus 4.7	0	0	0

Table 8: Canary audit over the 340-task test split. *Tasks w/ can.* = number of tasks whose output contains at least one canary token; *Copyable tok.* = count of emitted canary tokens whose prefix appears in a model-visible input (query or conversation context); *Hidden tok.* = emitted canary tokens absent from all visible inputs (the contamination signal). No system produced a hidden-canary match.

B Additional QueryDiagramEval Details

Deterministic fact extraction. Table 9 lists the structural facts parsed for each Mermaid family. Extractors are type-specific because Mermaid types encode different semantics: a sequence diagram is primarily participants and messages, a Gantt chart is tasks and sections, and a git graph is branches and commits. Feeding these facts to the auditor reduces counting errors and discourages visual hallucination by making the code’s actual contents explicit.

Diagram type	Extracted facts
flowchart	Nodes, edges, subgraphs
sequenceDiagram	Participants, messages with labels
classDiagram	Classes, relationships with types
erDiagram	Entities, relationships
stateDiagram	States, transitions
gantt	Tasks, sections, date format
gitGraph	Branches, commits
journey	Steps with satisfaction, sections
mindmap	Hierarchy tree
quadrantChart	Data points, axes, quadrant names
timeline	Events and time periods

Table 9: Type-specific structural facts extracted deterministically from candidate code (no LLM inference).

Parser accuracy audit. Because the fact block anchors the semantic auditor, its faithfulness to the

candidate code matters directly. We audited the type-aware parser against an independent regular-expression oracle on all 340 human-verified gold diagrams of the resplit benchmark (68 sources $\times$ 5 difficulty tiers). For each diagram we compared the parser’s element counts (participants, classes, entities) and its relation set (edges, messages, relationships) against the oracle; Table 10 reports per-type element-count exactness and relation recall for the five structural families.

The audit surfaced—and we fixed—several parser gaps that had depressed relation recall on two families: flowchart middle-label edges (A -- label --> B) and quoted/Unicode pipe labels (S -->| "ε" | A), which previously yielded zero edges; sequenceDiagram inline-activation shorthand (A->>B: msg); classDiagram relationships carrying multiplicity labels (A "1" -- "*" B); and erDiagram entities declared through attribute blocks or bearing :: style suffixes. Before these fixes, relation recall was 71% (flowchart), 88% (sequenceDiagram), and 72% (classDiagram), and entity-count exactness was 83% (erDiagram). After the fixes, element counts and relation recall both reach 100% against the oracle across all five structural families (Table 10).

Two caveats apply. First, this is an automated cross-check against a second deterministic oracle rather than a human element-by-element audit; correlated errors shared by both parsers would not be detected, so a full human-verified precision/recall audit across all eleven types remains future work. Second, the fact block is *anchoring evidence*, not the sole basis for scoring: the auditor prompt instructs the judge to inspect the code and rendered image directly when a fact is absent, so an occasional missed relation degrades to ordinary multimodal reading rather than a silent scoring error, and the Level 1 gates independently catch total parse failures.

Type	n	Element-count exact	Relation recall
flowchart	35	—	100%
sequenceDiagram	40	100%	100%
classDiagram	35	100%	100%
erDiagram	30	100%	100%
stateDiagram	25	—	100%

Table 10: Parser accuracy audit against an independent oracle on 340 human-verified gold diagrams. *Element-count exact* is the fraction of diagrams whose extracted element count (participants/classes/entities) exactly matches the oracle; *relation recall* is the fraction of oracle relations (edges/messages/relationships) recovered. Dashes mark families for which the oracle does not provide an independent element-count reference.

Rubric weighting. Table 11 gives the five dimensions used by the fact-grounded auditor. Adherence and completeness receive the highest weights because diagram-generation failures are usually semantic; readability and accessibility remain in the score but with lower weight so that a visually polished diagram cannot compensate for failing to satisfy content requirements. Weights were fixed a priori by rubric design and were not tuned on the annotation data.

Dimension	w_d	Evaluates
Adherence to prompt	0.30	Scope, constraints, style
Completeness	0.25	Required components present
Conceptual correctness	0.20	Logical relationships
Readability	0.15	Spacing, crossings, sizing
Accessibility	0.10	Contrast, color independence

Table 11: Fact-grounded audit rubric dimensions and weights.

Compilation gates in detail. Before semantic scoring, QDE extracts Mermaid code from fenced blocks, parses it with a type-aware AST parser, and renders it with a pinned Mermaid.js runtime. Parse failure caps the score at 25; render failure at 40 (empty code at 20); diagram-type mismatch at 60 (or 85 if content is otherwise correct). These gates are deliberately separated from the LLM auditor so that non-renderable or wrongly typed code cannot receive a high score merely because it appears plausible in text.

Reported metric suite. Table 12 separates deterministic Level 1 diagnostics from the Level 2 QDE score. We report render success and type accuracy

alongside QDE because they expose different failure modes: a model can score well semantically only after it produces valid, renderable, correctly typed code. All metrics in the table are reference-free.

Metric	Level	Det.	Ref.
Render success rate	1 (Compilation)	✓	—
Type accuracy	1 (Compilation)	✓	—
$S_{\text{fig}} / \text{QDE}$	2 (Semantic)	grounded	—

Table 12: QDE metric suite. “Grounded” denotes LLM auditing anchored by deterministic structural facts.

C Human Annotation Protocol

Because our central claim is that QDE aligns with human judgment, we document the annotation study in full: who annotated, how they were trained, what they saw, the exact rubric, adjudication, per-dimension agreement, and compensation. Annotation followed a fixed written guideline (v1.0, April 2026) issued to every annotator before work began.

Annotator pool and expertise. Annotations were produced by a contracted professional annotation team of six annotators (anonymised A01–A06). They were *technical but not diagram-generation experts*: the guideline required only that annotators be able to *read* Mermaid and judge whether a diagram satisfies a natural-language request—not to author Mermaid code. Baseline familiarity was established through a 30-minute Mermaid tutorial with ten worked examples and per-type syntax cheat-sheets covering all eleven diagram families. Three annotators (A01–A03) carried the full-coverage load ($\sim 2,480$ items each); the remaining three (A04–A06) rated stratified subsets (~ 930 each).

Training and calibration. Every annotator completed a mandatory calibration phase before any production annotation: (i) all annotators independently rated the *same* 20 calibration tasks; (ii) a one-hour calibration meeting discussed and resolved disagreements against reference anchors; and (iii) annotators re-rated 5 tasks to confirm alignment. Annotators could not begin the main phase until calibration was complete. During the main phase, tasks were presented in randomised order stratified by tier and type, with enforced rest breaks to limit fatigue effects, and annotators were instructed not to discuss individual items with one another.

Blinding. The metric-validation annotations (Task C) are *reference-blind*: annotators rated each model prediction purely against the user query and never saw the gold diagram, the source model identity, or the automatic quality bucket the item was drawn from. Non-rendering outputs were excluded upstream so that annotators always judged a visible diagram. This isolates query-satisfaction judgments from gold leakage and model-identity bias.

Rubric. Each prediction was scored on seven 1–5 dimensions plus structured error fields (Table 13). Dimensions C1–C6 were designed to mirror the automated auditor’s dimensions to enable per-dimension correlation; C0 (overall quality) was rated holistically as an independent gut judgment rather than a weighted average of C1–C6. Each dimension carried explicit 1/3/5 anchor descriptions in the guideline. In addition, annotators selected all applicable tags from a 15-way error taxonomy (C7; the basis for Appendix F.6) and listed missing (C8) and spurious (C9) elements as free text.

ID	Dimension	Scale / content
C0	Overall quality	1–5 holistic
C1	Readability	1–5
C2	Accessibility	1–5
C3	Adherence & constraints	1–5
C4	Completeness	1–5
C5	Conceptual correctness	1–5
C6	Factual/structural accuracy	1–5
C7	Error tags	multi-select (15-way taxonomy)
C8	Missing elements	free-text list
C9	Spurious elements	free-text list

Table 13: Human annotation rubric (Task C, reference-blind). C1–C6 mirror the automated auditor dimensions; C0 is an independent holistic rating.

Adjudication and item repair. For ordinal ratings the median of the primary ratings was used; items whose ratings spanned more than two points were flagged for discussion. For binary judgments, unresolved disagreement among the primary annotators was escalated to a senior annotator who made the final call. Separately, benchmark query–gold pairs (Task B) flagged by ≥ 2 annotators as low-faithfulness ($B1 \leq 2$) or insufficient ($B2 = \text{No}$) were repaired—query rewritten or gold corrected—or discarded by the project lead before the splits were frozen and any model evaluation began.

Per-dimension agreement. Table 14 reports inter-annotator agreement for every rubric dimension, not only holistic quality. Agreement was

computed over the multiply-annotated pool (297 items with four independent ratings; 1,800 rating pairs on the 1–5 scale). Semantic dimensions most tied to diagram structure show the strongest agreement—completeness ($\alpha=0.838$), holistic quality (0.797), adherence (0.795), conceptual correctness (0.757)—while the more subjective visual dimensions (accessibility 0.514; structural fidelity 0.528) are lower, partly due to ceiling effects (several annotators assigned near-uniform high scores). Mean absolute deviation between annotators is 0.30–0.45 on the five-point scale, i.e. annotators typically agree within about half a point. Holistic-quality agreement is stable across the five batches ($\alpha \in [0.69, 0.84]$) and pairwise annotator Spearman ranges 0.59–0.88 (mean 0.759, 30 pairs).

ID	Dimension	α	ICC	MAD
C0	Overall quality	0.797	0.795	0.45
C1	Readability	0.613	0.612	0.30
C2	Accessibility	0.514	0.513	0.33
C3	Adherence	0.795	0.793	0.45
C4	Completeness	0.838	0.837	0.33
C5	Conceptual correctness	0.757	0.755	0.31
C6	Structural fidelity	0.528	0.528	0.34

Table 14: Inter-annotator agreement per rubric dimension on the Task C pool (Krippendorff’s α , intraclass correlation, and mean absolute deviation on the 1–5 scale). Computed over 297 items with four independent ratings.

Compensation and ethics. Annotation was carried out by a contracted professional annotation team; annotators were compensated by their employer under the terms of that commercial engagement rather than per-task by the authors. Annotators rated only synthetic or model-generated diagram content paired with authored queries and were not asked to provide personal or sensitive information.

D Fact-Grounded Judge Prompts

This appendix reproduces the prompts used by the fact-grounded semantic-assessment stage of QUERYDIAGRAEVAL (S_{fg} , §3). The pipeline issues one system prompt (§D.1) plus a per-task user message that bundles the query, candidate code, deterministic structural facts (§D.2), and the rendered PNG (§D.3). $k=5$ independent samples are drawn from the judge with temperature 1.0 and aggregated by the rule of §3.2.

D.1 System prompt

System prompt (fact-grounded judge)

You are an expert diagram quality evaluator. You will assess how well a generated Mermaid diagram satisfies a user's request.

You are given:

1. The user's original query
2. The generated Mermaid code
3. A rendered image of the diagram (if available)
4. **Deterministic structural facts** extracted from the code – these are ground truth about what the code actually contains

Use the structural facts as anchoring evidence while performing semantic quality assessment.

Evaluation Process

Step 1: Requirement Extraction. Read the user query and enumerate every atomic requirement (R1, R2, ...). Aim for 10–30 granular requirements.

Step 2: Fact-Anchored Requirement Checking. For each requirement, check whether it is satisfied using BOTH:

- The structural facts (element lists, counts, type detection)
- Your own semantic analysis of the code and image

For each R_n, output one of {satisfied, missing, partial}. When the structural facts confirm an element exists, treat that as strong evidence. When they don't mention something, check the code and image yourself.

Step 3: Hallucination Detection. Check for elements in the diagram NOT requested. Use the structural facts to systematically verify each element has a corresponding requirement.

Step 4: Visual Quality Assessment. Score readability and accessibility from the rendered image (or code heuristics if no image).

Step 5: Structured Output. Return a JSON object with these exact fields:

```
{
  "requirements": [{"id": "R1",
    "text": "...",
    "status": "satisfied|missing|partial",
    "evidence": "structural_fact|semantic|image"}],
  "hallucinated_elements": [...],
  "error_counts": {"n_total_requirements",
    "n_satisfied", "n_missing", "n_partial",
    "n_hallucinated"},
  "readability": <0-5>, "accessibility": <0-5>,
  "adherence_to_prompt_and_constraints": <0-5>,
  "completeness": <0-5>,
  "conceptual_correctness": <0-5>,
  "explanations": [...]}
}
```

Scoring Rubric (apply mechanically from error counts; abbreviated)

adherence_to_prompt_and_constraints: 5 = 0 missing/partial, 0 hallucinated; 4 = 1 missing or 1–2 partial; 3 = 2 missing or 3+ partial; 2 = 3+ missing; 1 = major scope violation; 0 =

completely wrong.

completeness: 5 = all satisfied; 4 = 1 missing; 3 = 2 missing; 2 = 3–4 missing; 1 = 5+ missing or <50% satisfied; 0 = <25% satisfied.

conceptual_correctness: 5 = all logic correct; 4 = 1 wrong direction or minor logic error; 3 = 2 logic errors; 2 = fundamental misunderstanding; 1 = mostly wrong; 0 = completely wrong.

readability: 5 = clean layout; 4 = minor spacing; 3 = some overlap; 2 = significant issues; 1 = hard to read; 0 = unreadable.

accessibility: 5 = good contrast and labels; 4 = minor contrast issue; 3 = some concerns; 2 = poor contrast or tiny text; 1 = major issues; 0 = inaccessible.

D.2 Structural-fact summary

The structural-fact block is generated deterministically from the candidate code by a typed Mermaid parser; the auditor never sees the gold code. The block always includes parse status, render status, the detected diagram type, and code length; the remainder is type-specific.

Structural-fact template (verbatim)

STRUCTURAL FACTS (deterministic, ground-truth from code analysis):

Parse: OK | FAILED

Render: OK | FAILED

Detected diagram type: <type>

Code length: <n> lines

Type-specific block (excerpt):

Nodes (k): id1=label1, id2=label2, ...

Edges (k): src -> dst [label]

Sections / classes / participants / states / entities / events: ...

D.3 Per-task user prompt

Per-task user message

User Query

<query>

Generated Mermaid Code

```mermaid

<predicted code>

```

STRUCTURAL FACTS (deterministic, ...)

<fact summary from App. C.2>

Evaluate this diagram against the user's requirements. Use the structural facts as

ground truth for what the code contains.
Return ONLY the JSON object.

[image: data:image/png;base64,<...>]

D.4 Aggregation

For every S_{fg} call, the auditor’s per-dimension 0–5 ratings are mapped to a 0–100 score via the fixed rubric weights $w = (0.15, 0.10, 0.30, 0.25, 0.20)$ over (readability, accessibility, adherence, completeness, conceptual correctness). The final per-task QDE is the arithmetic mean of $k = 5$ independent samples; per-task standard deviation $\bar{\sigma}_k$ is reported in Table 2. Level 1 hard gates (parse/render/type) cap the score before averaging (§3.1).

E MermaidAgent Architecture

This appendix expands the main-paper description of MERMAIDAGENT (§5) into the full architecture used for the agentic rows of Table 2. The DiagramAgent baseline (Wei et al., 2024) is ported to the same pinned Mermaid backend with its original tool interface preserved, providing an apples-to-apples agentic comparison.

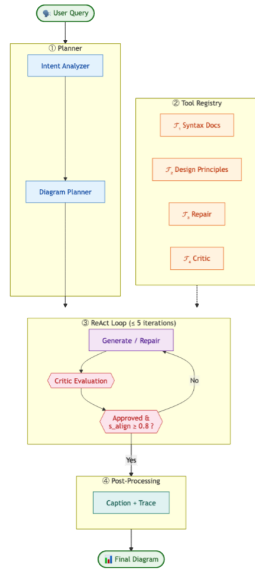


Figure 5: MERMAIDAGENT overview. A user query flows through the Planner (intent analysis and diagram plan), is grounded by four tools ($\mathcal{T}_1$ – $\mathcal{T}_4$), and enters an iterative ReAct loop that generates, critiques, and repairs candidate diagrams until the quality threshold ($s_{\text{align}} \geq 0.5$) is met.

Planner. Given a user query, the Planner executes two sequential LLM calls under structured output schemas. The *intent analysis* call maps the query to a closed-vocabulary diagram type (one of the eleven supported Mermaid varieties), estimated complexity, subject domain, ranked key requirements, detected ambiguities, and style constraints. The closed vocabulary bounds the search space and prevents hallucination of unsupported types. The *diagram plan* call then consumes the intent analysis and produces (i) a structural outline, (ii) key nodes/entities, (iii) edges/relationships, (iv) an optional styling approach, and (v) a justification of how the plan meets the complexity requirements. The plan is injected into the generation prompt as a *soft* constraint, not mechanically enforced, so that generative flexibility is preserved while intent alignment is maintained.

Tool registry. Four tools support the agent: two *knowledge tools* consulted once per request and two *action tools* invoked within the ReAct loop.

Documentation ($\mathcal{T}_1$). Given a diagram type, returns a 700–1,000-character authoritative syntax summary covering keywords, edge syntax, node shapes, and structural constraints. For unknown types it returns the full list of supported types, enabling graceful degradation. This grounding prevents the LLM from hallucinating syntax from outdated pre-training data.

Design principles ($\mathcal{T}_2$). Maintains a static, expert-curated knowledge base of over 100 design heuristics along four axes: (i) *general* principles—clarity, consistency, simplicity, readability (4 guidelines each); (ii) *type-specific* rules for all eleven diagram varieties; (iii) *styling*—colour usage, shape semantics, annotation placement; and (iv) *complexity-level* element-count targets for simple (3–7), intermediate (8–15), and advanced (≥ 16) diagrams. Per request, one query retrieves type-specific and general principles, and another retrieves complexity-level guidelines; both are injected into the generation prompt.

Repair ($\mathcal{T}_3$). Provides three utilities: *syntax validation* via a two-stage pipeline (lightweight structural checks for missing keywords, empty bodies, unbalanced delimiters, followed on pass by a full grammatical parse); *structure analysis* that counts diagram elements and identifies the diagram type; and *fix suggestion* via rule-based pattern matching on validation errors and structural counts to produce a prioritised list of repair suggestions.

Critic ($\mathcal{T}_4$). Performs a deterministic, multi-dimensional evaluation in five stages: (1) *syntax validity* (a syntax error classifies the diagram as poor quality and halves the alignment-score weight); (2) *type correctness* (a mismatch is recorded as a structural issue); (3) *requirement coverage* (Eq. 3); (4) *structural completeness* (diagrams with < 2 elements are flagged as too simple; those with > 50 elements trigger a decomposition recommendation; flowchart and sequence diagram types with zero edges are additionally flagged); and (5) *alignment scoring* (Eq. 4). For each key requirement r_i , the evaluator checks whether any content-bearing token (length > 3) appears in the lowercased diagram source:

$$s_{\text{cov}} = \frac{|\{r_i \in R \mid r_i \text{ is covered}\}|}{|R|}. \quad (3)$$

A coverage score below 0.5 triggers an improvement suggestion. The alignment score combines coverage with a syntax-weighted multiplier:

$$s_{\text{align}} = \text{clip}(s_{\text{cov}} \cdot w_{\text{syn}}, 0, 1), \quad (4)$$

$$w_{\text{syn}} = \begin{cases} 0.7 & \text{syntax valid,} \\ 0.3 & \text{otherwise.} \end{cases}$$

The approval decision is the conjunction

$$\begin{aligned} \text{approved} \iff & \text{syntax_valid} \wedge \text{type_match} \\ & \wedge |\text{design_issues}| \leq 1 \wedge s_{\text{cov}} \geq 0.7. \end{aligned} \quad (5)$$

ReAct loop. The core loop runs up to $N_{\text{max}} = 5$ iterations, each following a Thought $\rightarrow$ Act $\rightarrow$ Observe cycle. *Iteration 1* (initial generation) composes a prompt from the user query, intent analysis, diagram plan, syntax documentation ($\mathcal{T}_1$), and design principles ($\mathcal{T}_2$); the LLM returns Mermaid source and a brief explanation via a structured output schema, with markdown fences stripped automatically. Structural validation is applied immediately, and on failure a focused *syntax-only repair* sub-routine fires before critique. *Iterations 2– N_{max}* (repair) feed the LLM the current diagram, full critic feedback, user query, intent analysis, and original plan, and produce a repair plan listing issues, strategy, and concrete changes; this is followed by an optional syntax-only pre-repair and then a full repair whose prompt includes the repair plan and the first 1,000 characters of the syntax reference as a grounding guard. In syntax-only mode

the LLM is constrained to minimal edits—no structural redesign, preservation of node IDs, labels, and edges unless they directly cause the error—and outputs raw Mermaid markup with a recorded change list.

Termination and post-processing. After each iteration, $\mathcal{T}_4$ evaluates the diagram. The loop exits early when `approved = true` and $s_{\text{align}} \geq \tau$, with $\tau = 0.5$ by default; otherwise the last diagram is returned with a warning. Every step appends a record (action, output length, validity, alignment score, approval, issues addressed) to a trace log. Once the loop terminates, the agent prompts the LLM to produce a 2–3 sentence caption. The caption, final Mermaid source, iteration count, alignment score, and full trace are returned as the agent’s structured response.

F Additional Experimental Results

The main paper reports the headline comparison across model configurations. This section restores supporting analyses: dataset separation, per-type behaviour, fact-grounding ablations, tier effects, judge-model choice, cross-judge sensitivity, human error tags, and a qualitative example.

F.1 Per-Type Analysis

Table 15 breaks QDE down by diagram family. The pattern is heterogeneous: Claude-based systems are consistently strong across many types, while GPT reasoning produces isolated wins on state diagrams and quadrant charts. MERMAIDAGENT/Claude Opus 4.6 is the most broadly competitive, leading on flowcharts, ER diagrams, and journeys and staying close to the best model on Gantt charts. The hardest families are gitGraph and classDiagram for several configurations, consistent with their dense symbolic syntax and sensitivity to relation direction, branching, and class-edge semantics. The gold row is a strong human reference rather than a hard ceiling because QDE scores query satisfaction rather than exact agreement with the gold program.

Model	flow	seq	class	state	ER	gantt	git	jour	mind	quad	time
GPT-5.4	65.7	79.5	83.0	70.1	85.8	86.6	56.2	83.3	69.1	62.8	84.4
GPT-5.4 (reasoning)	57.9	88.2	63.5	92.0	74.1	88.4	68.3	87.6	65.0	97.1	78.2
Claude Opus 4.6	86.0	87.0	88.0	83.6	90.3	94.2	70.4	89.6	89.2	82.3	83.1
Claude Opus 4.6 (reason.)	86.1	93.6	84.9	67.5	86.7	89.5	86.0	89.4	82.3	83.8	86.3
DiagramAgent w/ GPT-5.4	78.4	88.2	76.2	79.6	81.7	83.7	68.1	88.0	70.3	60.1	76.9
DiagramAgent w/ Claude	81.2	84.3	79.4	77.1	83.4	85.0	70.5	86.8	74.2	65.0	79.3
MermaidAgent w/ GPT-5.4	65.5	75.7	61.5	69.3	75.0	80.5	76.3	83.9	65.4	77.4	86.7
MermaidAgent w/ Claude	89.5	87.2	86.6	85.4	90.5	94.0	79.8	90.0	86.2	89.3	79.2
Gold baseline	88.4	89.7	88.5	84.5	87.6	93.8	79.4	87.8	80.8	90.4	82.7

Table 15: QDE by diagram type on the test set, averaged across tiers.

F.2 Metric and Judge Ablations

Table 16 decomposes the validation result by human rating dimension. Correlations are strongest with adherence, conceptual correctness, and completeness—the dimensions most tied to structural diagram semantics. Readability and accessibility are positive but weaker, reflecting that human holistic scores weight semantic fidelity more heavily than visual polish.

Human dim.	$\rho(\text{dim}, S_{\text{fg}})$	95% CI
C1 (Readability)	0.316	[0.196, 0.426]
C2 (Accessibility)	0.256	[0.139, 0.371]
C3 (Adherence)	0.699	[0.620, 0.772]
C4 (Completeness)	0.582	[0.482, 0.668]
C5 (Conceptual)	0.654	[0.564, 0.733]
C0 (Holistic)	0.715	[0.632, 0.784]

Table 16: Spearman ρ between each human rating dimension and the holistic S_{fg} score on the validation pool ($N = 255$; fact-grounded judge Claude Opus 4.6).

Quality bucket (C0)	n	$\rho(\text{C0}, \text{QDE})$
Poor ($C0 < 2.5$)	24	0.244
Medium ($2.5 \leq C0 < 3.5$)	29	0.146
Good ($C0 \geq 3.5$)	202	0.567
All	255	0.715

Table 17: Per-bucket correlation stability on the validation pool ($N=255$). Buckets are defined on the holistic human scale ($C0 \in [1, 5]$): Poor < 2.5 , Medium $\in [2.5, 3.5)$, Good ≥ 3.5 . Within-bucket ρ is attenuated by range restriction and by a coarse, ceiling-heavy human scale (69/255 items rated exactly 5.0). QDE is most discriminative for broad quality separation—reliably distinguishing bad from good diagrams—which is the property most relevant to benchmark ranking.

Broad quality discrimination. Table 17 shows that QDE reliably separates poor from good diagrams, the property most relevant for benchmark ranking. Within-bucket correlations are lower, but this is expected: the human C0 scale is coarse and heavily tied at the ceiling (69 items at exactly 5.0),

which mechanically depresses within-bucket Spearman independently of QDE’s true resolution. The conclusions in Table 2 are also system-level means over 340 tasks rather than per-item rankings; aggregation averages out item-level noise, and the cross-judge study (Appendix F.5) confirms the resulting system ordering is stable (Kendall $\tau=0.905$, top four systems rank-identical).

Configuration	$\rho(\text{C0})$
S_{cot} (vanilla judge, no facts)	0.668
S_{fg} (generic facts)	0.688
S_{fg} (type-specific facts, $k = 5$)	0.715

Table 18: Fact-grounding ablation on the validation pool ($N = 255$). Type-specific structural facts contribute the largest single improvement (+0.027); generic structural facts add +0.020.

Fact-grounded judge	$\rho(\text{C0})$	$\bar{n}_{\text{err}}$ (std)
GPT-4.1	0.568	10.1 (12.3)
GPT-5.2	0.595	6.3 (6.7)
Claude Opus 4.6	0.715	5.1 (6.0)

Table 19: Judge model comparison on the validation pool. Claude Opus 4.6 achieves the strongest calibration and is used as the default judge.

F.3 Reference-Based Structural Baseline (GRAPHF1)

The reference-based structural score reported in Table 3 ($\rho=0.462$) is GRAPHF1, a fully deterministic (no-LLM) metric that scores a candidate diagram against its gold Mermaid code. It is an *in-house reimplementing inspired by* the graph-based diagram metric of Liang and You (2025), not a wrapper around their released code; differences are itemized at the end of this subsection. We specify it in full so it is reproducible.

Graph abstraction. Both the gold and candidate code are parsed by the same typed Mermaid parser used for the structural facts (§D.2) and reduced to a directed graph $G = (V, E)$, where V is a set of labeled nodes and $E \subseteq V \times V$ is a set of ordered (src, dst) edges. $\text{GRAPHF1} = 0.4 \cdot \text{NODEF1} + 0.6 \cdot \text{EDGEF1}$, weighting relational structure above node inventory.

Label normalization. Each node label is lower-cased, outer whitespace stripped, the characters `_` `-` `/` `\` mapped to spaces, and internal whitespace runs collapsed to single spaces. Two labels match *exactly* when their normalized forms are identical;

otherwise a *fuzzy* match fires when the normalized Levenshtein similarity $1 - \text{editdist} / \max(|a|, |b|) \geq 0.8$. Edge style (solid vs. dashed), relationship keywords, cardinality annotations, and edge labels are discarded—only endpoints and direction are compared.

Node alignment. Nodes are matched one-to-one by a greedy pass in three priority tiers: (1) exact normalized-label equality; (2) fuzzy Levenshtein similarity ≥ 0.8 ; (3) a fallback on normalized node *IDs*, which recovers cases where a model reuses gold identifiers but omits labels. Each gold node aligns to at most one candidate node. Matched pairs are true positives; unmatched gold nodes are false negatives; unmatched candidate nodes are false positives, yielding NODEF1.

Edge direction. Edges are ordered (src, dst) pairs, so *direction is scored*: a reversed edge is simultaneously a false positive and a false negative. Candidate edges are re-expressed in the gold node space via the node alignment (the raw candidate endpoints are also retained, to catch direct ID reuse) and then set-intersected with the gold edge set to compute EDGEF1. Undirected relations (e.g. some class/ER associations) are stored in their parsed textual order and thus treated as directed for scoring purposes—a known conservative approximation.

Non-graph diagram types. The parser maps all eleven families to the (V, E) abstraction, but coverage is uneven and this is a deliberate limitation of the baseline. The seven graph-structured families populate both sets: flowchart (nodes/edges), classDiagram (classes / relationship edges), erDiagram (entities / relationship edges), sequenceDiagram (participants / message edges), stateDiagram (states / transition edges), gitGraph (commits / branch-commit edges), and mindmap (the one hierarchical list type retained as a full graph via parent-child edges). The remaining list-structured families collapse to a label set with an *empty* edge set, so on these types GRAPHF1 degenerates to NODEF1 over labels: gantt and journey reduce to their task/section labels (dates, durations, ordering, and journey scores are ignored), timeline to its period/event labels (chronological order ignored), and quadrantChart to its point labels (x/y coordinates and quadrant placement ignored). This structural blindness on non-graph types is one reason GRAPHF1 correlates far more weakly with human ratings than the fact-

grounded auditor, which consumes type-specific facts (§D.2) that capture exactly these ordering, date, and coordinate signals.

Relation to Liang and You (2025). We adopt their central idea—node and edge F1 over a graph abstraction with fuzzy label matching—but the implementation is our own and differs in four ways: (i) we operate directly on parsed Mermaid source rather than on graphs extracted from rendered images; (ii) we use the normalization and three-tier alignment above rather than their embedding-based matcher; (iii) we fix the 0.4/0.6 node/edge weighting; and (iv) we add type-specific parsing for Mermaid’s non-flowchart families. Consequently GRAPHF1 is reported only as a reference-based structural yardstick for contextualizing QDE; its absolute values are *not* numerically comparable to the original DiagramEval scores.

F.4 Difficulty-Tier Effects

Table 20 reports changes between successive query tiers. The average pattern is not strictly monotonic: T1→T2 is nearly flat on average, while T2→T3 and T3→T4 are negative. Adversarial or noisy wording alone is less damaging than indirect descriptions requiring structural inference. The large drop for GPT-5.4 reasoning from T3 to T4 shows that extended reasoning does not automatically make Mermaid generation robust to distractors, whereas Claude reasoning is comparatively stable across tiers.

Model	$\Delta T1-2$	$\Delta T2-3$	$\Delta T3-4$	$\Delta T1-4$
GPT-5.4	+7.9	-2.2	-3.8	+1.9
GPT-5.4 (reasoning)	-6.2	+1.8	-11.2	-15.6
Claude Opus 4.6	+1.1	+3.3	-4.0	+0.4
Claude Opus 4.6 (reasoning)	-0.2	+0.2	-1.7	-1.7
DiagramAgent w/ GPT-5.4	+2.2	-7.1	-0.9	-5.8
DiagramAgent w/ Claude	+3.1	-5.5	-1.2	-3.6
MermaidAgent w/ GPT-5.4	-6.2	-1.8	+8.6	+0.6
MermaidAgent w/ Claude	-0.5	-2.9	-2.2	-5.6
<i>Model mean</i>	+0.2	-1.8	-2.1	-3.7

Table 20: QDE changes between successive query tiers; negative values indicate degradation.

F.5 Cross-Judge Sensitivity

To check whether the choice of fact-grounded judge meaningfully changes our conclusions, we draw

a stratified random sample of 175 (model, task) pairs (25 per system, 7 systems) and re-score every pair with GPT-4.1 as an alternative fact-grounded judge. Prompts, structural-fact construction, image attachment, and $k=5$ self-consistency are kept identical. Table 21 reports per-system means under each judge.

System	Claude	GPT-4.1	Δ	r_C/r_G
Claude Opus 4.6 (reasoning)	89.81	96.75	+6.94	1/1
MermaidAgent (Claude)	83.77	95.25	+11.49	2/2
Claude Opus 4.6	81.18	91.13	+9.94	3/3
GPT-5.4 (reasoning)	79.93	87.41	+7.48	4/4
GPT-5.4	75.70	81.86	+6.15	5/6
DiagramAgent (GPT-5.4)	74.36	83.78	+9.42	6/5
MermaidAgent (GPT-5.4)	68.58	77.25	+8.66	7/7

Table 21: Cross-judge sensitivity ($n=25$ per system, 175 total). Δ = mean QDE under GPT-4.1 minus mean under Claude; r_C/r_G = system rank under each judge.

GPT-4.1 is systematically more lenient (every system gains 6–11 points) but the relative ordering is highly stable: Kendall $\tau = 0.905$ (Spearman $\rho = 0.964$, Pearson $r = 0.966$, $n = 7$). The single rank swap involves two systems whose Claude means differ by only 1.34 points. Per-task agreement is $\rho = 0.698$ (Pearson $r = 0.927$, $n = 175$). Together with Table 19, this rules out a self-enhancement bias large enough to reorder the systems in Table 2.

F.6 Human Error Taxonomy

Table 22 gives the human error-tag distribution underlying the discussion in §7. The most common errors are missing edges, missing nodes, layout/readability problems, label mismatches, hallucinated nodes, and factual errors. Syntax and type mistakes are relatively uncommon, especially compared with semantic omissions, which is why render success and type accuracy are insufficient as standalone metrics.

Error type	T1	T2	T3	T4	MT	All
Syntax error	0.0	8.2	0.0	1.6	1.1	2.0
Type misclassification	4.6	4.9	1.3	0.0	1.1	2.2
Missing nodes	18.5	26.2	20.8	15.6	15.4	19.0
Missing edges	23.1	32.8	22.1	12.5	23.1	22.6
Hallucinated nodes	18.5	14.8	19.5	4.7	12.1	14.0
Hallucinated edges	15.4	8.2	13.0	10.9	11.0	11.7
Inverted logic	1.5	9.8	3.9	9.4	1.1	4.7
Label mismatch	16.9	18.0	18.2	15.6	7.7	14.8
Ordering error	9.2	11.5	9.1	6.2	6.6	8.4
Wrong cardinality	1.5	3.3	9.1	3.1	3.3	4.2
Over-specification	13.8	14.8	22.1	6.2	11.0	13.7
Layout/readability	18.5	23.0	11.7	14.1	11.0	15.1
Factual error	6.2	18.0	18.2	9.4	8.8	12.0

Table 22: Human error-tag distribution by tier (% of annotated predictions exhibiting each error; $n = 358$).

F.7 Qualitative Example

To make the failure modes in Tables 15 and 22 concrete, we walk through one held-out Tier-3 *sequenceDiagram* task in the rail-signaling domain (task V4_T2_010526_180431_dd0d90), representative of typed, workflow-oriented diagrams that appear in industrial documentation. The gold Mermaid program contains canary-mutated identifiers (XQV03xx_*) injected at construction time; the candidate is produced by MERMAIDAGENT backed by GPT-5.4 with no access to the gold.

User query (T3)

I need a visual representation of our onboard equipment startup and connection procedure. Here’s how the process works:

1. The train operator (user of the XQV0323 system) initiates an ignition request to the onboard Handheld Terminal (HHT).
2. Upon receiving the ignition request, the HHT performs an internal initialization sequence.
3. Immediately after initialization, the HHT runs a series of self-tests to verify its own operational readiness.
4. Once self-tests are complete, the HHT establishes a connection to the communication network (this is categorized under XQV0324 connectivity).
5. The HHT then sends a second request through the network specifically to connect to the Radio Block Centre (RBC).
6. The network relays the connection request onward to the RBC, formally requesting the establishment of that link on behalf of the operator.

The key participants are: the train operator, the HHT, the Network, and the RBC. Number the interactions so each step is referenceable. The HHT’s initialization and self-test steps are internal and don’t involve other parties.

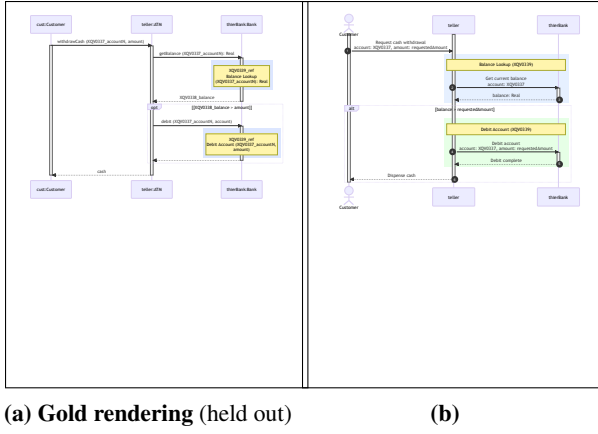


Figure 6: Rendered gold vs. prediction for the qualitative example. Both diagrams pass Level-1 gates; the candidate preserves participants and the six-step ordering but paraphrases canary-tagged labels and adds three Note over annotations.

Gold Mermaid (held out)

```
sequenceDiagram
    autonumber
    actor XQV0323_User
    participant HHT
    participant Network
    actor RBC

    XQV0323_User->>HHT: XQV0322_Request ignition
    HHT->>HHT: Initialize
    HHT->>HHT: Start self-tests
    HHT->>Network: XQV0324_Connect to network
    HHT->>+Network: XQV0324_Connect to RBC
    Network->>RBC: XQV0322_Request the connection to RBC
```

MERMAIDAGENT w/ GPT-5.4 prediction

```
sequenceDiagram
    autonumber
    actor TO as Train Operator
    participant HHT as Handheld Terminal (HHT)
    participant NET as Communication Network
    participant RBC as Radio Block Centre (RBC)

    Note over TO,HHT: Startup initiated by the operator
    TO->>HHT: 1. Ignition request

    Note right of HHT: Internal HHT processing only
    HHT->>HHT: 2. Internal initialization sequence
    HHT->>HHT: 3. Run self-tests for readiness

    Note over HHT,NET: XQV0324 connectivity
    HHT->>NET: 4. Establish connection to network

    Note over HHT,NET,RBC: XQV0324 RBC link establishment
    HHT->>NET: 5. Request connection to RBC via network
    NET->>RBC: 6. Relay RBC connection request

    deactivate HHT
```

What QDE captures. The candidate parses, renders, and is correctly typed, so all Level 1 gates pass. The fact extractor confirms that all four gold participants are present (TO/HHT/NET/RBC) and that the six-step ordering—ignition → self-tests → network → RBC—is preserved, which the fact-grounded auditor scores high on *conceptual cor-*

rectness and *completeness*. Two cost-bearing differences remain: (i) the candidate replaces canary-tagged labels (XQV0322_*) with descriptive paraphrases, and (ii) it adds three Note over annotations beyond the gold scope, which the auditor flags as soft hallucinations. The candidate also turns RBC’s actor role into a generic participant, a small type-specific drift that surface-only metrics would miss but that the structural facts surface explicitly. The example illustrates why QDE separates renderability from semantic fidelity: a high-quality but imperfect candidate receives strong adherence/completeness credit while still being penalised for label rewording and over-annotation, matching the dominant error categories in Table 22.